

1. 论文

GAP: A novel Generative context-aware Prompt-tuning method for relation extraction

1.introduction

fineTuning方法是关系抽取的主流，但这有一个小缺点，即微调的目标和预训练的目标不一致，这导致了模型无法充分发挥他在预训练阶段学到的知识。

为了解决这个问题，[prompt-tuning](#)方法出现，核心思想是与其让模型做出改变去适应任务，不如把任务变成模型熟悉的样子。

例如 Barack Obama [MASK] Honolulu，直接让模型预测mask

这就让任务和模型的预训练对齐了，即变成了他的来时路。然而现有的方法主要有以下缺陷：

- 1.人工设计模板：obviously要人工给数据集设计prompt模板。
- 2.也不是完全对齐：设计出来的模板在预训练的语料中几乎很少出现，所以模型对他们其实也并不熟悉。
- 3.没有充分利用上下文信息，例如"President Barack Obama was born in the beautiful city of Honolulu, Hawaii"，president暗示了Obama是个人，city暗示了Honolulu是个地方，这些[trigger words](#)被忽略了。

2.方法

2.1 任务

给定一个句子: Barack Obama was born in Honolulu.

令这个句子为 X ，则有 $X = \{w_1, w_2, \dots, w_n\}$

同时有 $w_{sub} = \{w_s, w_{s+1}, \dots, w_{s+t}\}$

$w_{obj} = \{w_o, w_{o+1}, \dots, w_{o+j}\}$

即给定一个句子 X ，以及他的[主语实体](#)和[宾语实体](#)，去预测关系 y

2.2 Context-aware Pre-extractor

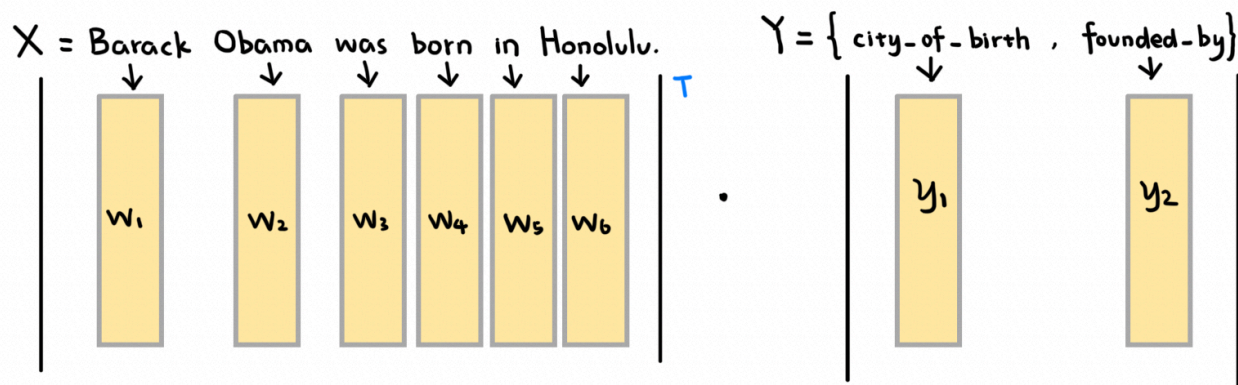
对应先前缺点的第三点，没有充分利用上下文信息，因此首先设计extractor。实际上，由于prompt-tuning的本质是一种[生成式方法](#)，因此其实会比传统的分类方法要发散，[要不确定](#)，因为选择的可能性多了，因此设计这个模块，是相当于先判断一下。

2.2.1 Attention over Attention

$X = \{w_1, \dots, w_n\}$ 即一个句子中的每个单词

$Y = \{y_1, \dots, y_m\}$ 即待预测关系集合

$M_{ij} = w_i^T \cdot y_j$, 即用点积衡量每个词和每个关系的相关性



2.2.2 从两个视角做 softmax

做点积之后的结果 would be like:

Barack · birth	Barack · founded
Obama · birth	Obama · founded
was · birth	was · founded
born · birth	born · founded
in · birth	in · founded
Honolulu · birth	Honolulu · founded

① M_{c2l} 矩阵, 即上下文到标签的相关性矩阵

我们首先固定住列, 也就是固定关系, 看看这个句子的哪个词最能支持这个关系?

$$\therefore M_{c2l} = \text{softmax}[M_{(1,t)}, M_{(2,t)}, \dots, M_{(|X|,t)}]$$

即把每一列做 softmax.

② M_{l2c} 矩阵, 标签到上下文相关性矩阵

obviously 另一种视角就是关注这个词最支持哪个关系?

2.2.3 融合两个视角

$$\beta = \frac{1}{n} \sum_{t=1}^{|L|} M_{c2l}^{(t)} = \left[\begin{array}{c} \uparrow \text{求平均} \end{array} \right]$$

∴ β 代表的是每个关系和句子整体的相似度。

Barack · birth
Obama · birth
was · birth
born · birth
in · birth
Honolulu · birth

Barack · founded
Obama · founded
was · founded
born · founded
in · founded
Honolulu · founded

接着再把 β 和 M_{l2c} 做点积, 再通过 softmax 得到预提取概率分布 α

$$\alpha = \text{softmax} \left(\frac{M_{l2c} \cdot \beta^T}{\sqrt{d_\beta}} \right)$$

d_β 是 β 维度, 经典 scale 操作

Barack · birth	Barack · founded
Obama · birth	Obama · founded
was · birth	was · founded
born · birth	born · founded
in · birth	in · founded
Honolulu · birth	Honolulu · founded

6×2

·

整句与 birth
整句与 founded

2×1

即 =

6×1

该操作的直觉是
用宏观视角加权筛选
每个词对各关系的局部
支持度

2.3 Pre-trained prompt generators

Why?

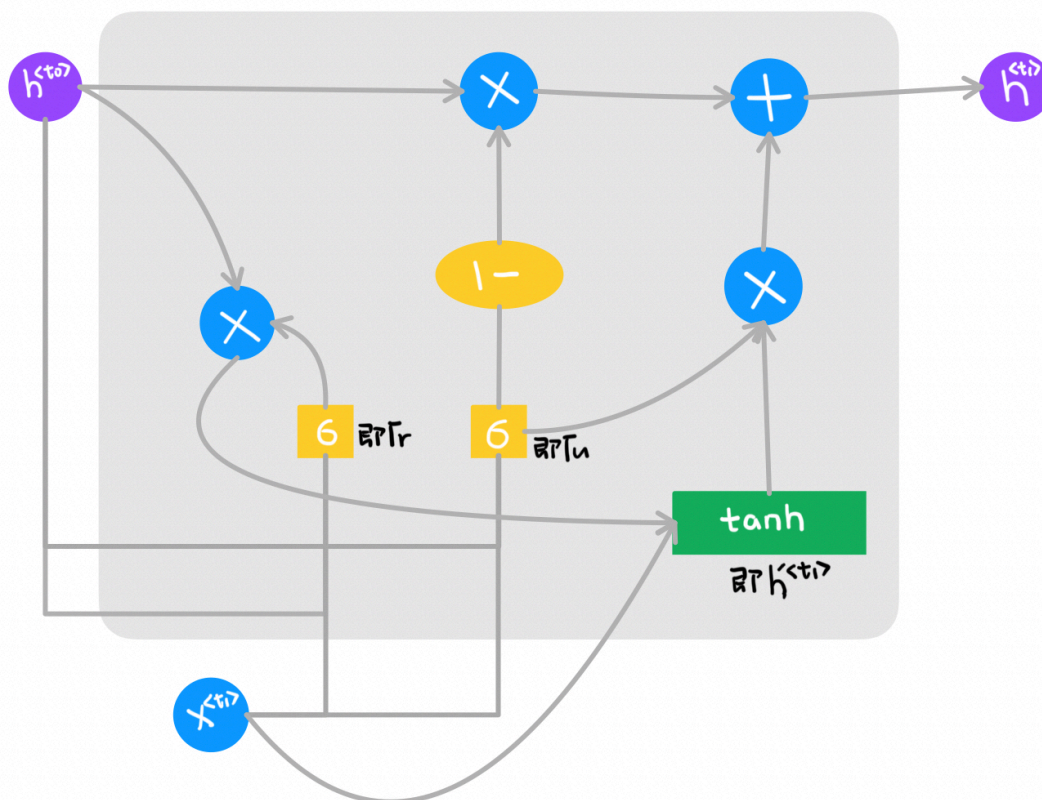
传统的prompt-tuning需要人工设计模板，GAP的结局方案是，训练一个专门的神经网络，让他自动从句子中读出最有用的触发词，接着把这些触发词压缩成prompt token嵌入进去。这个prompt generator的骨架是一个GRU，作者专门解释了为什么不用transformer：单层transformer非线性能力较弱；多层参数太多容易过拟合，不如选择GRU，在序列建模上更擅长捕捉局部的上下文依赖而且轻量。

2.3.1 GRU

$$\begin{aligned} \Gamma_r &= \sigma(W_r[h^{<to>}, x^{<ti>}] + b_r) \\ \Gamma_u &= \sigma(W_u[h^{<to>}, x^{<ti>}] + b_u) \end{aligned} \rightarrow h^{<ti>} = \tanh(W_h[\Gamma_r \cdot h^{<to>}, x^{<ti>}] + b_h)$$

hidden state candidate

$$h^{<ti>} = (1 - \Gamma_u) \cdot h^{<to>} + \Gamma_u \cdot h^{<ti>}$$
$$\hat{y}^{<ti>} = g(W_y h^{<ti>} + b_y)$$



2.3.2 回到论文，编码整个句子

$$H = \text{GRU}_{\text{enc}}(w) = \{h_1, h_2, \dots, h_n\}$$

把我们句子中的每个单词送入一个单层双向GRU，得到了一系列中间状态编码，每个小h都包含了这个词左边上下文的意思和右边上下文的意思，所以H是对我们整个句子语义的全面编码

2.3.3 用实体解码

解码器通常以一个特殊符号<start>作为第一步输入，GAP不这样做，而是把主语实体Wsub和宾语实体Wobj的嵌入，分别作为解码器的起始输入。这样做的目的是，让解码器找到和Wsub和Wobj最相关的线索词语，而不是随便什么词语。

$$T_{sub} = \sum_{p=s}^{s+i} \varphi(w_p) \cdot \text{Emb}(w_p) \rightarrow \text{原始向量}$$

→ 出现频率, whole vocab

$$T_{obj} = \sum_{q=0}^{o+j} \varphi(w_q) \cdot \text{Emb}(w_q)$$

这样就得到了所谓的头实体和尾实体的词嵌入，作为encoder的输入

• 注意力机制

$$e_i^t = V^T \tanh(W_h h_i + W_s s_t + b_{\text{attn}})$$

$$a_i^t = \text{softmax}(e_i^t)$$

$$c^t = \sum_i a_i^t h_i$$

公式1计算的是第t步中，解码器对第i个单词的关注度得分，送入的是头实体和尾实体，送进去之后，根据解码器状态 s_t 和之前的编码器得到的H，再用Bahdanau注意力机制，算出注意力分布 a^t ；再把他和状态H加权求和，得到上下文向量 c^t 。也就是这一步的句子的综合信息。

$$P_{\text{vocab}}(w') = \text{softmax}(V'(V[s_t, c_t] + b) + b')$$

看不懂，但反正这个公式就是头尾词嵌入送进去以后，哪个单词是下一个的概率最大，就是probability，针对的是整个词表

2.3.4 指针机制

$$p_{\text{aen}} = \sigma(W_c^T c_t + W_s^T s_t + W_x^T x_t + b_{\text{en}})$$

当前上下文

当前 decoder 状态

当前输入

$$P_{\text{vocab}}(PW_t) = P_{\text{gen}} \cdot P_{\text{vocab}}(w') + (1 - P_{\text{gen}}) \cdot \sum_{i: w_i = w'} \alpha_i$$

实际上，2.3.3就已经可以生成词语了，针对的是整个词表，但是为了防止decoder输出无意义的字符例如<unk>，导致白白浪费一个位置，所以引入这种指针机制。

所以如果 P_{gen} 接近于1，decoder就倾向于从词表里面输出一个新词；反之就会按照原句子的注意力分布，直接复制一个词语出来。这就保证了，如果模型生成的触发词语不好，那就退而求其次，直接从原句子选一个词语出来作为触发词。有点像交叉熵。

总之这一部分是为了构建自动化的提示词模板，因此你需要选择一些所谓的触发词，即一些和头实体、尾实体最相近的词语。前面就是如何选这些触发词的数学原理。

方法就是利用encoder-decoder结构：

1. encoder是GRU，且送进去的是头实体和尾实体本身的词嵌入

2. decoder也是GRU，在解码生成的时候为了不浪费位置，不让模型输出没用的触发词，加入了指针机制。如果真的输出的是没用的，那就直接从原句子中复制一个词语过来。

2.3.5 把触发词嵌入

2.3.4生成了 - 堆触发词 $\{PW_1, \dots, PW_m\}$ 后

采用了一种简洁的 context Embedding 方式

即和encoder方式一样，用频率加权

$$[\text{Head Prompt}] = \sum_{i=1}^n \varphi(PW_i^{\text{sub}}) \cdot \text{Emb}(PW_i^{\text{sub}})$$

$$[\text{Tail Prompt}] = \sum_{i=1}^n \varphi(PW_i^{\text{obj}}) \cdot \text{Emb}(PW_i^{\text{obj}})$$

最终的Prompt模板如下：

Barack Obama [Head Prompt] [Mask] [Tail Prompt] Honolulu

所以最终的模板就是“头本身 + 他的触发词 + mask + 他的触发词 + 尾本身”
做了一堆事情就是为了选出触发词。

2.4 In-domain Adaptive Pre-training

上述的prompt generator在训练的时候做的是文本摘要任务（CNN DailyMail数据集）why?

答：上述任务的核心是从词表中找出最能揭示关系的线索词，所以有点像在做[提炼、汇总](#)工作，这就和摘要任务的本质不谋而合。

所以我們也需要这个In-domain pre-training模块，核心任务就是在做正式的关系抽取任务之前，先恶补一下目标领域的知识。prompt-tuning对这种领域不同[尤为敏感](#)，因为他高度依赖预训练时期学习到的知识，所以必须要恶补。

2.4.1 BM25

$$BM_{25}(Q, d) = \sum W_i \cdot R(q_i, d)$$

GAP从三个大型公开语料库去检索（SST2、DBPedia和AGNews），用了这个经典的BM25算法，可以自动化帮你找出与你的查询Q最相近的一些文本片段，计算成本远低于神经网络。所以他只是关键词检索，不理解语义。

2.4.2 N-gram掩码

	Barack	Obama	was	born	in	Honolulu
BERT采样率	15%	15	15	15	15	15
BERT采样结果	Barack	Obama	was	[M]	in	Honolulu
GAP率	30	30	30	30	30	30
GAP结果	Barack	[M]	thanks	birth	in	Honolulu

- 1.提高掩盖码率到30%
- 2.随机选择N值，即选中了一个词之后，从「1, 2, 3」中随机选一个值，选到几就同时掩码几个，可以让模型更好理解短语级别的语义。
- 3.且如果前一个词已经被掩码，下一个掩码率强制设置为0。
- 4.此外，选中之后的掩码策略不是一定是mask，80%是mask，10%是被随机替换，10%被替换为同义词。

不过作者在消融实验里承认该模块的实验性能提升非常有限。SemEval提升0.0%，TACREV提升0.1%。可能是上面选的那三个公开语料库不够大，且未必包含目标数据集所需的领域专业知识，所以如果换成例如pubmed文献，效果可能会好很多。

额，不知道为什么一开始不直接选择生物医学领域语料。然后是对RoBERTa-large进行训练。

2.5 Joint Contrastive Loss

在prompt-tuning框架里，模型有两个输出需要被监督训练。第一个是【cls】token的输出，用来做传统的[关系分类](#)，另一个是【mask】token的输出，即需要预测这个位置应该填写什么词语。所以需要两种不同的损失函数来监督。模型用的还是RoBERT。

2.5.1 构建输入

Prompt(x) = P_{sub} [Header-Prompt] [Mask] [Tail-Prompt] P_{obj} → 2.3.5得到的

$$x_{in} = [CLS] \times [SEP] \text{prompt}(x) [SEP]$$



2.5.2 两种损失

$$y_{cls} = \text{softmax}(M([CLS]|X_{in})) + \alpha$$

$\downarrow$
 MLP, 输出类别

$\rightarrow$ BERT输出, 全局语义
 $\rightarrow$ 最开始的初步判断结果
 $\rightarrow$ 给最终结果压缩

$$L_{CE} = \text{Cross-Entropy}(y_{cls}, y_i)$$

$$\downarrow$$

$$L = -\sum y_i \log p_i = -\sum y_i \log y_{cls}, \text{ 多分类交叉熵}$$

$$y_{mask} = \text{softmax}(M([Mask]|X_{in}))$$

这里的[Mask]是一个词向量
 即预测这个位置填什么单词
 所以不能用交叉熵损失
 不是离散标签类别

$$L_{CT} = \frac{1}{N} \ln \frac{\exp(s(y_{mask}, y^+) / \tau)}{\exp(s(y_{mask}, y^+) / \tau) + \sum \exp(s(y_{mask}, y^-) / \tau)}$$

$\rightarrow$ 正确关系标签嵌入
 $\downarrow$
 错嵌

s()是余弦相似度，有些眼熟的公式，有点像比例占比，能把你往正例拉拢，从负例推开。温度参数tau控制拉拢力度，越小越激进，越去区别人。

最终的损失不是两者直接相加，而是

$$L = L_{CE} + (1 + KL(y_i, y_{cls})) \cdot L_{CT}$$

KL是KL散度，若两者预测一致，则KL约等于0，则两种损失权重一样，即预提取模块和预训练模型都能正确判断；
 而如果两则预测不一致，说明这是一个困难样本，KL会远大于0，所以Lct比重会大大增加，模型会投入更多学习资源去学习

2. 实验

数据集	模型	best epoch	mapped exact	eval loss
ChemProtSent	B4 semantics-only	9	0.901083	0.084065
ChemProtSent	B5 dual-view concat	17	0.896106	0.083922
ChemProtSent	v1	14	0.895932	0.098628
CDRItra	B5 dual-view concat	14	0.775451	0.251782
CDRItra	v1	10	0.776028	0.238254

加入信息筛选模块性能几乎没有提升，正在改善参数重新跑。